

Apuntes del Curso: Computación Paralela y Distribuida

Prof. José Fiestas

12 de agosto de 2026

U1.6. DAG (Directed Acyclic Graphs)

Modelo DAG

Los diagramas acíclicos dirigidos (Directed Acyclic Graphs, DAG) constituyen uno de los modelos más útiles para representar y analizar algoritmos paralelos. Permiten visualizar qué operaciones pueden ejecutarse simultáneamente y cuáles deben esperar que se completen tareas anteriores (dependencias).

Un DAG es un grafo

$$G = (V, E)$$

donde:

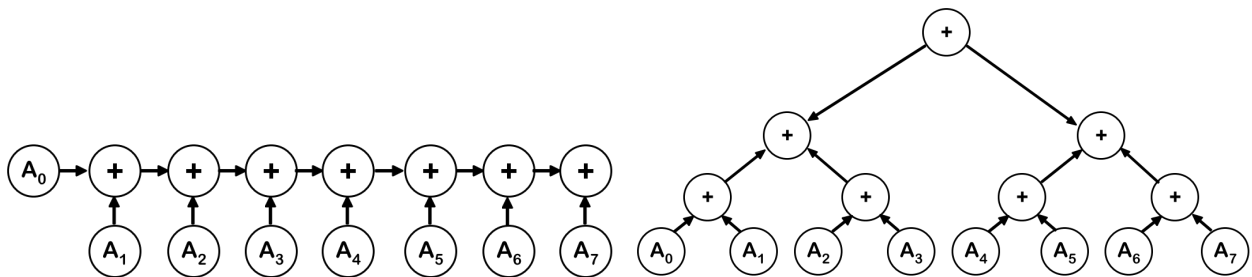
V: conjunto de vértices o nodos

E: conjunto de aristas dirigidas

No existen ciclos dirigidos

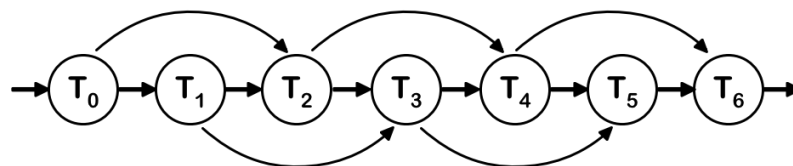
En computación paralela interpretamos normalmente:

- vértices representan **tareas** (instrucciones simples o en bloques)
- aristas representan **canales de comunicación** (dependencias)



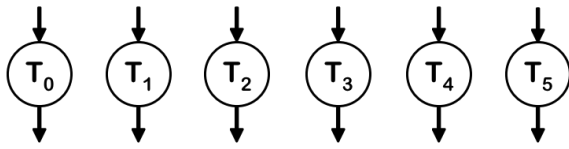
Podemos clasificar a los algoritmos de la siguiente forma:

- **Secuenciales**, no pueden ser paralelizados porque todas las tareas tienen dependencias en

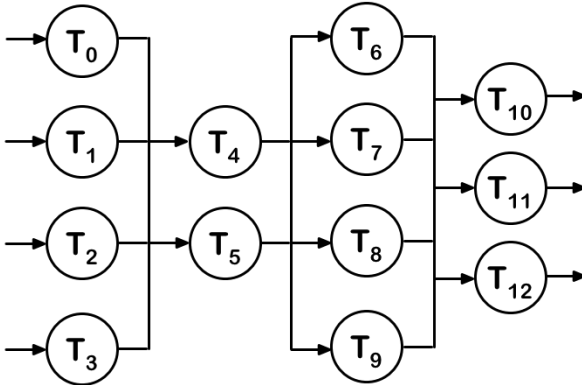


tareas previas

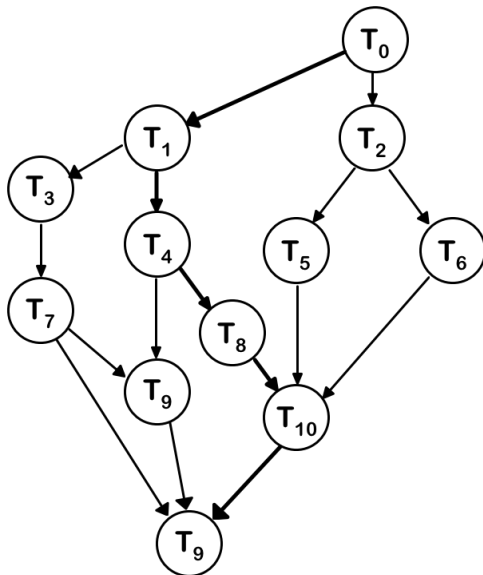
- **Paralelos**, donde todas las tareas pueden ser ejecutadas simultáneamente



- **SPA** (Secuencial-Paralelo) El algoritmo está separado en niveles, que se ejecutan en paralelo, pero los cuales tienen una forma secuencial de ejecución.



- **NSPA** (No-Secuencial-Paralelo) El algoritmo no contiene ninguno de los patrones anteriores



Modelo de costo DAG

El **scheduling** o 'plan de ejecución' de un DAG (V, E) , asigna a cada nodo i :

- un **tiempo de ejecución** t_i , tal que si $(i, i + 1) \in E$, el tiempo acumulado $T_{i+1} = T_i + t_{i+1}$.
Si las tareas son $O(1)$, $T_{i+1} = T_i + 1$
- un **procesador** p_i , $i \in \{0, \dots, p - 1\}$
En algunos casos los nodos de input corresponden a $t_0 = 0$

El algoritmo paralelo correspondiente se ejecuta en

$$T_\infty = \text{MAX}(\sum t_i)$$

, que es la longitud del plan de ejecución, llamada **span**

El **trabajo** (W) se define como la cantidad total de operaciones (instrucciones). Es decir, la complejidad secuencial (T_s)

Se busca que el DAG tenga un **trabajo** (mínimo) y un **span** (profundidad) mínima

El **trabajo** equivale al tiempo secuencial (optimo): $W(n) = T_s^*(n)$

En paralelo, p procesos ejecutaran por lo menos T_s/p unidades de tiempo. Es decir

$$T_p \geq T_s/p$$

Esto nos permite definir el **speedup** (velocidad) de un proceso en paralelo como:

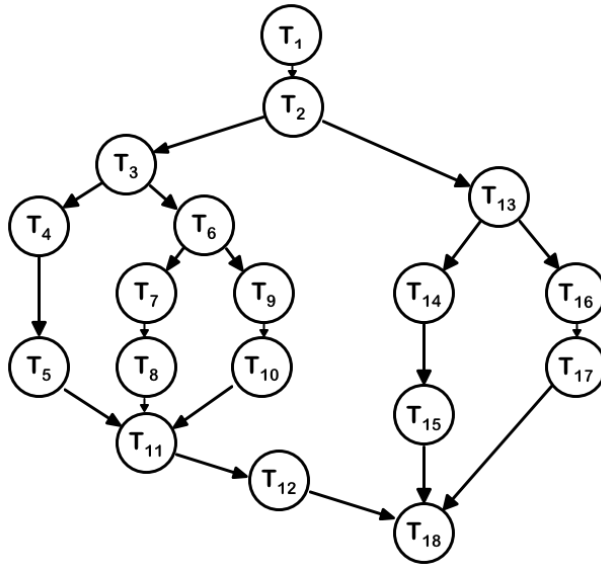
$$S = \frac{T_s}{T_p}$$

Un speedup igual (o proporcional) a p es un speedup lineal optimo, ya que $S = \frac{T_s}{T_p} \leq p$
Para el **span**, o camino critico del DAG, tenemos que

$$T_p \geq T_\infty$$

Ejemplo 01

Determine span y trabajo en el siguiente DAG si cada tarea tiene complejidad constante, $O(1)$



$W=18$ (numero total de tareas, equivale a complejidad secuencial)

$T_{\infty} = 9$

Ejemplo 2: función recursiva

Considere la función :

$$f(x) = x, x \leq 1$$

$$f(x) = f(x-1) + f(x-2), x > 1$$

i.e. $(a,b)=(f(x-1) \parallel f(x-2))$

Determine el span y trabajo

pasos	tareas
0	a
1	b, c
2	d, e
3	f
4	g

Trabajo (W):

$$W(n)=1, \text{ si } n \leq 1$$

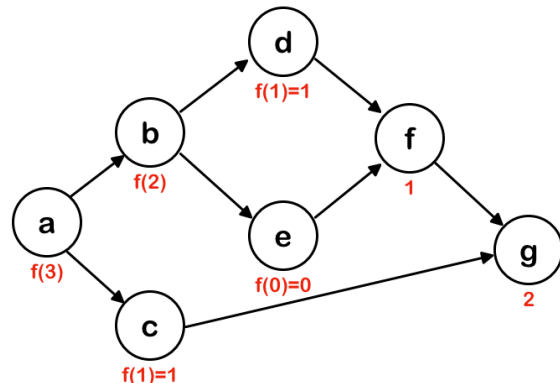
$$W(n)= W(n-1)+W(n-2)+1$$

Span:

$$T_{\infty}(n)=1, \text{ si } n \leq 1$$

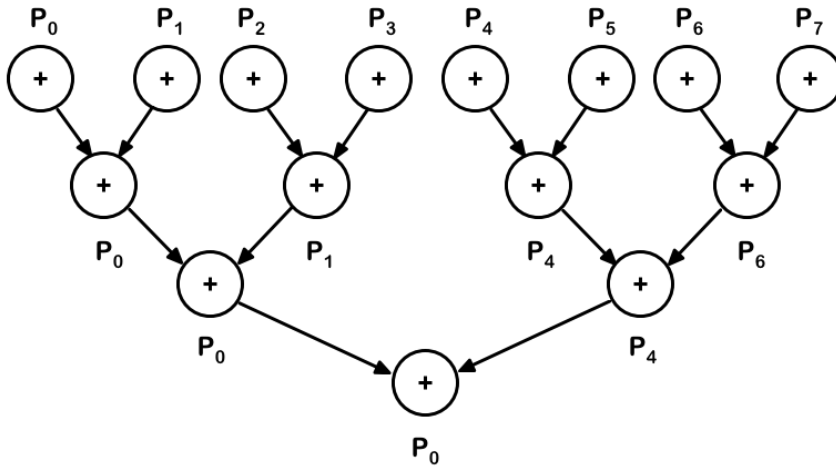
$$T_{\infty}(n)= \text{MAX}(T_{\infty}(n-1), T_{\infty}(n-2))+1$$

La solución recursiva permite obtener una expresión para T_{∞} .

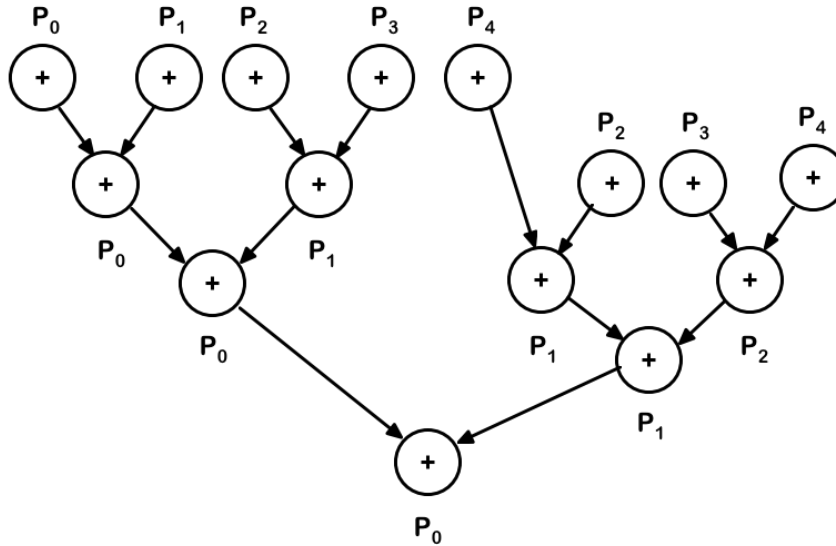


Ejemplo 3: Suma de números

1. Construya un DAG eficiente para la suma de n enteros en paralelo, si se tiene $n=16$ y $p=8$ (procesadores disponibles)
2. Compare el DAG con el caso $n=16$, $p=5$
3. Determine T_s , $T_\infty(n)$, $W(n)$, $S(n)$ y la clasificación del algoritmo según Flynn en cada caso



a)



b)

c) Determine T_s , $T_\infty(n)$, $W(n)$, $S(n)$ en cada caso

Contabilizando cantidad de pasos en cada DAG, se obtiene para el caso

a) $T_s(n=16) = 15$, $T_\infty(n=16)=4$, $W(n=16) = 15$, $S(n)=15/4$, $E(n)=15/4/8=0.47$

b) $T_s(n=16) = 15$, $T_\infty(n=16)=5$, $W(n=16) = 15$, $S(n)=15/5$, $E(n)=15/5/5=0.60$

Se utiliza la misma operación en cada nodo, lo que permite el cálculo por el conteo de aristas. Esto no es posible si las operaciones entre nodos difieren.

Se observa un menor speedup y una mayor eficiencia en el segundo caso. Esto se explica por el mayor aprovechamiento de recursos (procesos).

0.1. Ejemplo 4

Diagrama el DAG correspondiente al siguiente código

```
1 double a[N], b[N], c[N], v=0.0, w=0.0;
2 T1(a, &v);
3 T2(b, &w);
4 T3(b, &v);
5 T4(c, &w);
6 T5(c, &v);
7 T6(a, &w);
```

Las funciones leen y modifican ambos argumentos.

Determine T_s , $T_\infty(n)$, $W(n)$, $S(n)$ del algoritmo paralelo asumiendo una complejidad constante de las tareas dadas.

Ahora recalcule T_s , $T_\infty(n)$, $W(n)$, $S(n)$ si la complejidad de T1, T2, T5 es $O(n)$, mientras que $T3(n)=O(n \log n)$, $T4(n)=O(\log n)$ y $T6(n)=O(n^2)$

Referencias

- [1] Joseph Jája *An introduction of parallel algorithms*. University of Maryland
- [2] Fayez Gebali *Algorithms and Parallel Computing* Wiley, 2011
- [3] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.